

COMPUTER NETWORKS

Ex.No.5	Experiments on Packet capture tool: Wireshark
Date	

AIM:

To understand the features of wireshark as a packet capture tool and understand **encapsulation of information at various layers of a Protocol stack**.

WIRESHARK

Wireshark is a free, open-source network packet analyzer used for network troubleshooting, analysis, and security auditing. It lets you capture and interactively browse live data traffic passing through a network interface:

Work done by Wireshark:

- Capture network traffic
- Decode packet protocols using dissectors
- Define filters – capture and display
- Watch smart statistics
- Analyze problems
- Interactively browse that traffic

Wireshark used for:

- Network administrators: troubleshoot network problems
- Network security engineers: examine security problems
- Developers: debug protocol implementations
- People: learn **network protocol internals**

Getting Wireshark

Wireshark can be downloaded for Windows or macOS from its official website. For

Linux or another UNIX-like system, Wireshark will be found in its package repositories. For Ubuntu, Wireshark will be found in the Ubuntu Software Center.

Download Wireshark from the following link

<https://www.wireshark.org/download.html>

Procedure to Capture Packets Using Wireshark

Step 1: Download and install **Wireshark** on your computer.

Step 2: Open the **Wireshark** application.

Step 3: From the home screen, select the **network interface** (e.g., Wi-Fi or Ethernet) through which your computer is connected to the network.

Step 4: Click the **Start Capturing Packets** button (blue shark fin icon) or double-click the selected interface.

Step 5: Generate network traffic by opening a website, sending a ping, or browsing the Internet.

Step 6: Observe the captured packets appearing in real time in the Packet List pane.

Step 7: To view only specific packets, enter a **display filter** (e.g., `http, tcp, udp, icmp, dns`) in the filter bar and press **Enter**.

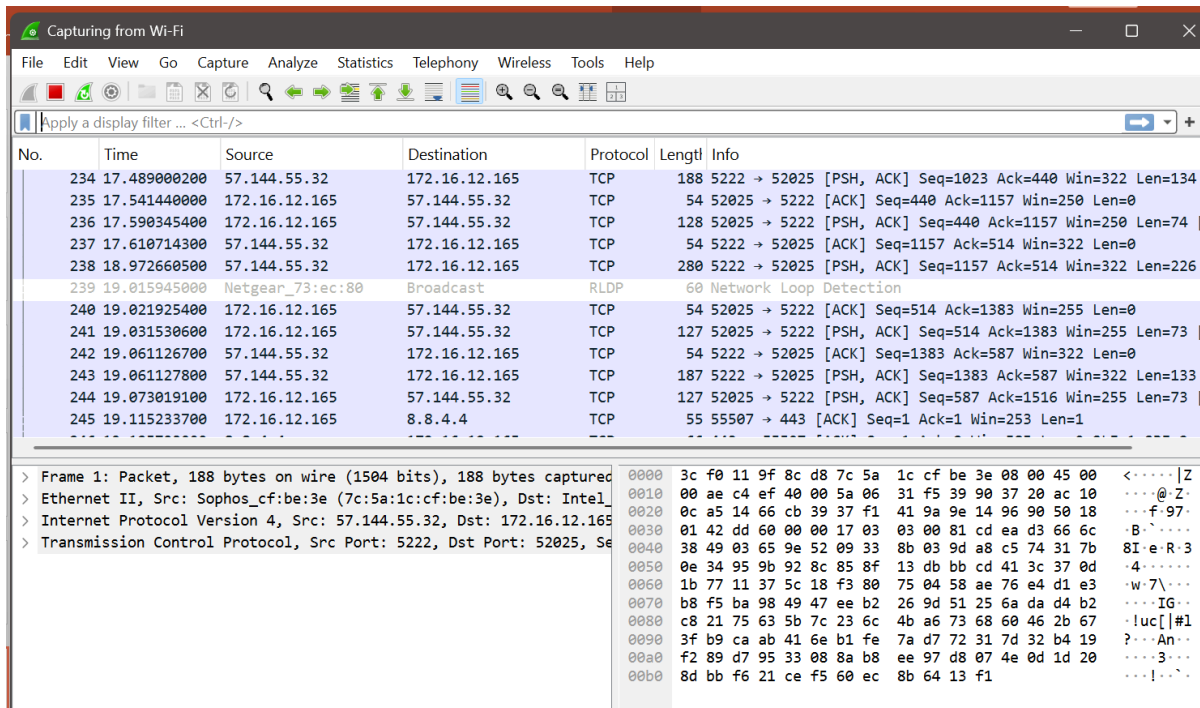
Step 8: Click on any packet to view its detailed information in the **Packet Details** pane.

Step 9: Expand the protocol layers (Ethernet, IP, TCP/UDP, etc.) to examine packet contents.

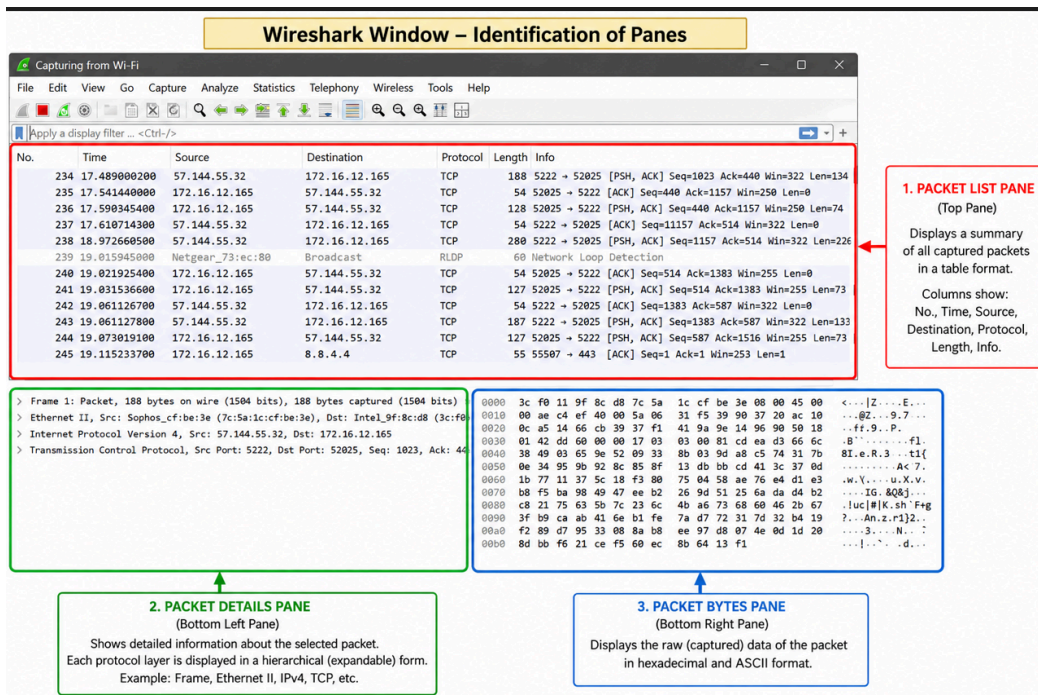
Step 10: Click the **Stop** button (red square) to stop packet capture.

Step 11: To save the captured packets, go to **File** → **Save As**, choose a location, and save the file in **.pcapng** (or **.pcap**) format.

Sample Capturing Packets from Wireshark: Sample output



Different Panes in Wireshark:



1. Packet List Pane (Top Pane)

- Displays a summary of all captured packets.
- Each row represents one captured packet.

- Shows details such as **Packet Number**, **Time**, **Source**, **Destination**, **Protocol**, **Length**, and **Info**.

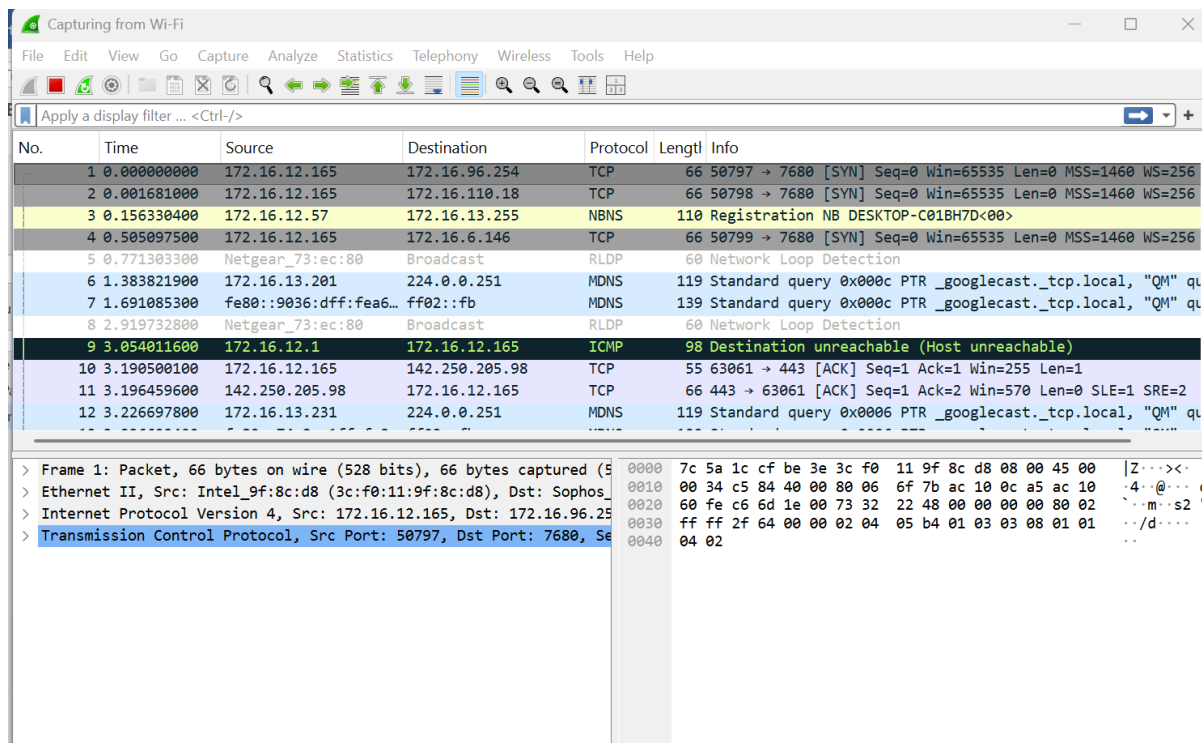
2. Packet Details Pane (Middle Pane)

- Displays detailed information about the selected packet.
- Information is organized into protocol layers such as **Frame**, **Ethernet**, **IP**, **TCP/UDP**, etc.
- Each protocol layer can be expanded to view more details.

3. Packet Bytes Pane (Bottom Pane)

- Displays the raw data of the selected packet.
- Data is shown in **hexadecimal** format on the left.
- The corresponding **ASCII** representation is shown on the right.

Note: These three panes work together to help users analyze captured network packets efficiently in Wireshark.



Color Coding:

In Wireshark, **packet colors** help you quickly identify different types of network traffic. Based on your screenshot, the colors indicate the following:

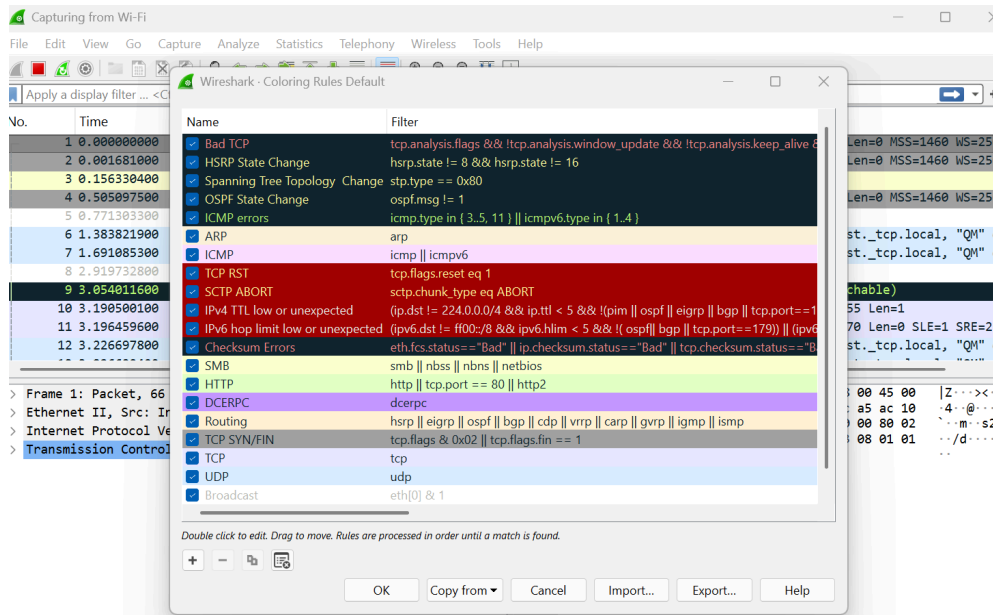
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	172.16.12.165	172.16.96.254	TCP	66	50797 → 7680 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256
2	0.001681000	172.16.12.165	172.16.110.18	TCP	66	50798 → 7680 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256
3	0.156330400	172.16.12.57	172.16.13.255	NBNS	110	Registration NB DESKTOP-C01BH7D<00>
4	0.505097500	172.16.12.165	172.16.6.146	TCP	66	50799 → 7680 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256
5	0.771303300	Netgear_73:ec:80	Broadcast	RLDP	60	Network Loop Detection
6	1.383821900	172.16.13.201	224.0.0.251	MDNS	119	Standard query 0x000c PTR _googlecast._tcp.local, "QM" qu
7	1.691085300	fe80::9036:dff:fea6...	ff02::fb	MDNS	139	Standard query 0x000c PTR _googlecast._tcp.local, "QM" qu
8	2.919732800	Netgear_73:ec:80	Broadcast	RLDP	60	Network Loop Detection
9	3.054011600	172.16.12.1	172.16.12.165	ICMP	98	Destination unreachable (Host unreachable)
10	3.190500100	172.16.12.165	142.250.205.98	TCP	55	63061 → 443 [ACK] Seq=1 Ack=1 Win=255 Len=1
11	3.196459600	142.250.205.98	172.16.12.165	TCP	66	443 → 63061 [ACK] Seq=1 Ack=2 Win=570 Len=0 SLE=1 SRE=2
12	3.226697800	172.16.13.231	224.0.0.251	MDNS	119	Standard query 0x0006 PTR _googlecast._tcp.local, "QM" qu

> Frame 1: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 0
 > Ethernet II, Src: Intel_9f:8c:d8 (3c:f0:11:9f:8c:d8), Dst: Sophos_...
 > Internet Protocol Version 4, Src: 172.16.12.165, Dst: 172.16.96.254
 > Transmission Control Protocol, Src Port: 50797, Dst Port: 7680, Seq=...

Color	Protocol/Traffic	Description
Light Gray	TCP	Indicates normal TCP packets such as SYN and ACK packets.
Light Yellow	NBNS (NetBIOS Name Service)	Used for NetBIOS name registration and name resolution on a local network.
White	RLDP (Network Loop Detection)	Represents Network Loop Detection Protocol packets used by network devices.
Light Blue	MDNS (Multicast DNS)	Used for local network service discovery (e.g., Chromecast, printers, Apple devices).
Dark Green / Black with Green Text	ICMP	Indicates Internet Control Message Protocol packets such as Destination Unreachable , Ping Request , or Ping Reply .
Light Purple / Lavender	TCP	Represents another category of TCP packets (e.g., ACK or established connection traffic) based on Wireshark's default coloring rules.

To view exactly what the color codes mean, click **View > Coloring Rules**. You can also customize and modify the coloring rules from here, if you like.

Sample Output:



Sample Captures:

- If there is no interesting network traffic on your own network, you can use **sample capture files**.
- Download a sample capture file and open it by selecting **File** → **Open** in Wireshark.
- Browse to the downloaded file and click **Open** to analyze the captured packets.
- You can also save your own captured packets for future analysis by selecting **File** → **Save**.
- Wireshark saves capture files in **.pcap** or **.pcapng** format.

Simple Procedure:

1. Download a sample capture file.
2. Open Wireshark.
3. Click **File** → **Open**.
4. Select the sample capture file.
5. Analyze the captured packets.
6. To save your own capture, click **File** → **Save**.

You can also save your own captures in Wireshark and open them later. Click **File** > **Save** to save your captured packets

Filtering Packets:

- 🎬 Close unnecessary applications before capturing packets.
- 🎬 This reduces unwanted network traffic.

- 🎬 Capture only the traffic related to the program you want to analyze.
- 🎬 Without filters, thousands of packets may be captured.
- 🎬 Filters help narrow down the packet list.
- 🎬 This makes packet analysis faster and easier.
- 🎬 Wireshark provides **Capture Filters** and **Display Filters** to simplify traffic analysis.

Sample filter applied for TCP:

Filtered Packet List

No.	Time	Source	Destination	Protocol	Length	Info
1	0.001	192.168.1.10	142.250.183.78	TCP	74	50234 → 443 [SYN]
2	0.005	142.250.183.78	192.168.1.10	TCP	74	443 → 50234 [SYN, ACK]
3	0.006	192.168.1.10	142.250.183.78	TCP	66	50234 → 443 [ACK]
4	0.020	192.168.1.10	142.250.183.78	TCP	1514	HTTP GET Request
5	0.055	142.250.183.78	192.168.1.10	TCP	1514	HTTP 200 OK

Display Filters in Wireshark:

- 🎬 Click **Analyze → Display Filters** to open the list of available display filters.
- 🎬 Choose a **predefined filter** from the default list.
- 🎬 You can **create your own custom filters**.
- 🎬 Save custom filters for **future use**.
- 🎬 Display filters make it easier to find specific packets.
- 🎬 For advanced filter syntax, refer to the **official Wireshark Display Filter documentation**.

Example: Create and Save a Custom Filter

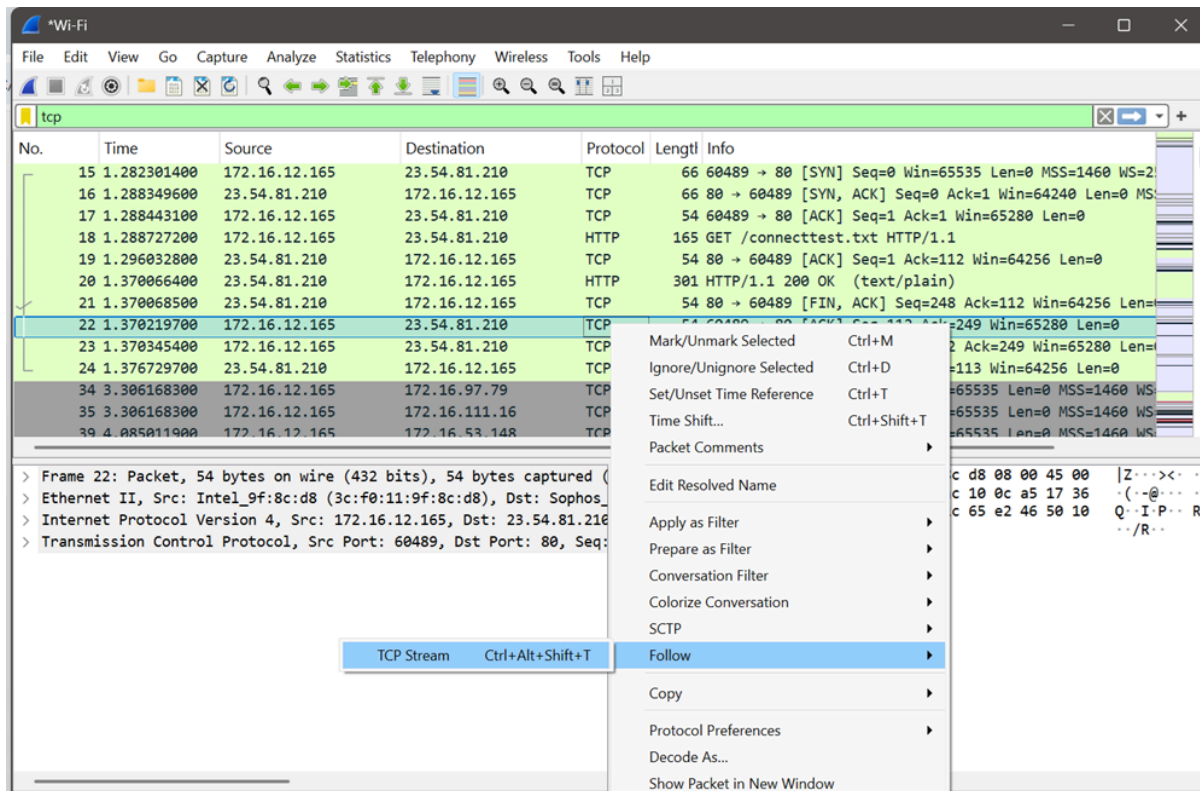
Suppose you frequently analyze **DNS packets**.

1. Click **Analyze → Display Filters**.
2. Click the **+** (**Add**) button.
3. Enter:
 - **Filter Name:** DNS Packets
 - **Filter Expression:** dns
4. Click **OK**.

Follow TCP Stream in Wireshark

1. Right-click on a **TCP packet**.
2. Select **Follow → TCP Stream**.
3. Wireshark displays the **complete conversation** between the client and the server.
4. This helps you understand the **data exchanged** during the communication.
5. You can also choose **Follow → UDP Stream** or other available protocol streams, if supported.

- This feature is useful for analyzing web requests, application data, and troubleshooting network communication.



Sample Output:

No.	Source	Destination	Protocol	Info
35	192.168.1.10	93.184.216.34	TCP	50321 → 80 [SYN]
36	93.184.216.34	192.168.1.10	TCP	80 → 50321 [SYN, ACK]
37	192.168.1.10	93.184.216.34	TCP	50321 → 80 [ACK]
38	192.168.1.10	93.184.216.34	HTTP	GET /index.html
39	93.184.216.34	192.168.1.10	HTTP	HTTP/1.1 200 OK

Inspecting Packets:

- Click a **packet** in the Packet List pane.
- The packet details are displayed in the **Packet Details** pane.
- Expand each protocol layer (Ethernet, IP, TCP/UDP, etc.) to view more information.
- View the raw packet data in the **Packet Bytes** pane.
- Inspecting packets helps analyze **source and destination addresses, protocol information, ports, and packet contents.**

Sample Output:

*Wi-Fi						
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help						
tcp.stream eq 0						
No.	Time	Source	Destination	Protocol	Length	Info
15	1.282301400	172.16.12.165	23.54.81.210	TCP	66	60489 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256
16	1.288349600	23.54.81.210	172.16.12.165	TCP	66	80 → 60489 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=
17	1.288443100	172.16.12.165	23.54.81.210	TCP	54	60489 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
18	1.288727200	172.16.12.165	23.54.81.210	HTTP	165	GET /connecttest.txt HTTP/1.1
19	1.296032800	23.54.81.210	172.16.12.165	TCP	54	80 → 60489 [ACK] Seq=1 Ack=112 Win=64256 Len=0
20	1.370066400	23.54.81.210	172.16.12.165	HTTP	301	HTTP/1.1 200 OK (text/plain)
21	1.370068500	23.54.81.210	172.16.12.165	TCP	54	80 → 60489 [FIN, ACK] Seq=248 Ack=112 Win=64256 Len=0
22	1.370219700	172.16.12.165	23.54.81.210	TCP	54	60489 → 80 [ACK] Seq=112 Ack=249 Win=65280 Len=0
23	1.370345400	172.16.12.165	23.54.81.210	TCP	54	60489 → 80 [FIN, ACK] Seq=112 Ack=249 Win=65280 Len=0
24	1.376729700	23.54.81.210	172.16.12.165	TCP	54	80 → 60489 [ACK] Seq=249 Ack=113 Win=64256 Len=0

> Frame 20: Packet, 301 bytes on wire (2408 bits), 301 bytes captured		0000	3c f0 11 9f 8c d8 7c 5a 1c cf be 3e 08 00 45 00	<.....
> Ethernet II, Src: Sophos_cf:be:3e (7c:5a:1c:cf:be:3e), Dst: Intel_		0010	01 1f 32 53 40 00 40 06 e5 c8 17 36 51 d2 ac 10	..2S@.@
> Internet Protocol Version 4, Src: 23.54.81.210, Dst: 172.16.12.165		0020	0c a5 00 50 ec 49 2c 65 e1 4e 10 0c 52 74 50 18	...P.I..N..RtP..
> Transmission Control Protocol, Src Port: 80, Dst Port: 60489, Seq:		0030	01 f6 4b 1f 00 00 48 54 54 50 2f 31 2e 31 20 32	..K...HT TP/1.1 2
> Hypertext Transfer Protocol		0040	30 30 20 4f 4b 0d 0a 43 6f 6e 74 65 6e 74 2d 4c	00 OK..Content-L
> Line-based text data: text/plain (1 lines)		0050	65 6e 67 74 68 3a 20 32 32 0d 0a 44 61 74 65 3a	ength: 2 2..Date:
		0060	20 54 68 75 2c 20 33 30 20 4a 75 6c 20 32 30 32	Thu, 30 Jul 202

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tcp.stream eq 0

No. Time

15 1.282301400

16 1.288349600

17 1.288443100

18 1.288727200

19 1.296032800

20 1.370066400

21 1.370068500

22 1.370219700

23 1.370345400

24 1.376729700

> Frame 20: Packet, 301 bytes on wire (2408 bits), 301 bytes captured (2408 bits) on interface \Device\NPF{...}

> Ethernet II, Src: Sophos_cf:be:3e (7c:5a:1c:cf:be:3e), Dst: Intel_9f:8c:d8 (3c:f0:11:9f:8c:d8)

> Internet Protocol Version 4, Src: 23.54.81.210, Dst: 172.16.12.165

> Transmission Control Protocol, Src Port: 80, Dst Port: 60489, Seq: 1, Ack: 112, Len: 247

> Hypertext Transfer Protocol

> Line-based text data: text/plain (1 lines)

0000 3c f0 11 9f 8c d8 7c 5a 1c cf be 3e 08 00 45 00 <.....|Z ...>..E..

0010 01 1f 32 53 40 00 40 06 e5 c8 17 36 51 d2 ac 10 ..2S@.@...6Q...

0020 0c a5 00 50 ec 49 2c 65 e1 4e 10 0c 52 74 50 18 ...P.I..N..RtP..

0030 01 f6 4b 1f 00 00 48 54 54 50 2f 31 2e 31 20 32 ..K...HT TP/1.1 2

0040 30 30 20 4f 4b 0d 0a 43 6f 6e 74 65 6e 74 2d 4c 00 OK..Content-L

0050 65 6e 67 74 68 3a 20 32 32 0d 0a 44 61 74 65 3a enght: 2 2..Date:

0060 20 54 68 75 2c 20 33 30 20 4a 75 6c 20 32 30 32 Thu, 30 Jul 202

0070 36 20 30 39 3a 31 39 3a 33 34 20 47 4d 54 0d 0a 6 09:19: 34 GMT..

0080 43 6f 6e 74 65 6e 74 2d 54 79 70 65 3a 20 74 65 Content- Type: te

0090 78 74 2f 70 6c 61 69 6e 0d 0a 43 61 63 68 65 2d xt/plain ..Cache-

00a0 43 6f 6e 74 72 6f 6c 3a 20 6d 61 78 2d 61 67 65 Control: max-age

00b0 3d 33 30 2c 20 6d 75 73 74 2d 72 65 76 61 6c 69 =30, mus t-revali

00c0 64 61 74 65 0d 0a 41 63 63 65 70 74 2d 52 61 6e date..Ac cept-Ran

No.: 20 - Time: 1.370066400 - Source: 23.54.81.210 - Destination: 172.16.12.165 - Protocol: HTTP - Length: 301 - Info: HTTP/1.1 200 OK (text/plain)

☒ Show packet bytes Layout: Vertical (Stacked)

Close Help

Flow Graph in Wireshark

Definition:

A **Flow Graph** visually represents the sequence of communication between different devices (hosts) in a network. It makes it easier to understand how packets are exchanged.

How to Open Flow Graph

1. Open a packet capture (.pcap) file.
2. Click **Statistics** → **Flow Graph**.
3. A Flow Graph window appears showing the communication between hosts.

What the Graph Shows

- **Vertical lines** represent different network devices (hosts).
- **Horizontal arrows** represent packets sent between hosts.
- **Time** is shown from **top to bottom**.
- Each arrow is labeled with the **protocol** and **packet information**.

Example:

Yes. Below is a **sample Flow Graph output** similar to what you would see in Wireshark after selecting **Statistics** → **Flow Graph**.

Sample Output: Flow Graph

Time	Client (192.168.1.10)	Server (93.184.216.34)
0.000	----->	DNS Query
0.015	<-----	DNS Response
0.030	----->	TCP SYN
0.035	<-----	TCP SYN-ACK
0.040	----->	TCP ACK
0.050	----->	HTTP GET /index.html
0.120	<-----	HTTP/1.1 200 OK
0.125	<-----	HTML Page Data
0.150	----->	TCP FIN
0.155	<-----	TCP ACK
0.160	<-----	TCP FIN
0.165	----->	TCP ACK

Wireshark Capture Options:

- ☐ Go to capture ☐ option
- ☐ Select stop capture automatically after 100 packets.
- ☐ Then click Start capture.

Step 1: Select the Active Interface

In your screenshot, choose the interface that is currently connected to the network.

- **If you are using Wi-Fi**, click **Wi-Fi**.
- **If you are using a LAN cable**, click **Ethernet**.

Tip: The active interface usually shows a moving graph or increasing traffic. In your screenshot, **Wi-Fi** is likely the correct choice if you are connected wirelessly.

Step 2: Go to the Options Tab

Click the **Options** tab at the top.

You will see settings similar to this:

☒ Stop capture automatically after

Step 3: Set the Packet Limit

Check the option:

☒ Stop capture automatically after

Select:

100 Packets

Creating a Filter to display only DHCP packets and inspect the packets.

Procedure

1. Open **Wireshark**.
2. Select the **Ethernet (Local Area Connection)** or **Wi-Fi** interface.
3. Click **Capture** → **Options**.
4. Select **Stop capture automatically after 100 packets**.
5. Click **Start** to begin packet capture.
6. Generate DHCP traffic by disconnecting and reconnecting the network, or by running:
7.

```
ipconfig /release  
ipconfig /renew
```
8. After the capture stops, type the following in the **Display Filter** bar:

dhcp
(If dhcp does not work, use bootp.)
9. Press **Enter** or click **Apply** to display only DHCP packets.
10. Click any DHCP packet to inspect its details.
11. Click **File** → **Save As**, enter a filename, and save the captured packets as a **.pcapng** file.

Create a Filter to display only IP/ICMP packets and inspect the packets.

Procedure

Step 1: Check if ping is working

1. Press **Windows + R**.
2. Type **cmd** and press **Enter**.
3. Type:

ping 8.8.8.8
4. Press **Enter**.

What do you see?

- If you see:
Reply from 8.8.8.8...
then the ping is working.
- If you see:
Request timed out
or
Ping request could not find host...
tell me exactly what message appears.

Step 2: Check whether Wireshark is capturing

When you click **Start Capture**, do you see the packet count increasing continuously?

- **Yes** → Wireshark is capturing.
- **No** → The wrong network interface is selected.

Step 3: Verify the filter

After capturing packets, clear any existing filter and type:

`icmp`

If the filter bar turns **green**, the filter is valid.

If it stays green but shows **0 packets**, then no ICMP packets were captured.

Step 4: Easier alternative

Instead of using `ping`, try this:

1. Start Wireshark.
2. Open **Google Chrome**.
3. Visit www.google.com.
4. Stop the capture.
5. In the Display Filter, type:

`dns`

or

`tcp`

6. Press **Enter**.

These filters almost always show packets because web browsing generates DNS and TCP traffic.

Sample Output (ICMP Filter)

Display Filter:

icmp

Packet List

No.	Time	Source	Destination	Protocol	Length	Info
15	2.345	192.168.1.10	8.8.8.8	ICMP	74	Echo (ping) request
16	2.356	8.8.8.8	192.168.1.10	ICMP	74	Echo (ping) reply
17	3.347	192.168.1.10	8.8.8.8	ICMP	74	Echo (ping) request
18	3.359	8.8.8.8	192.168.1.10	ICMP	74	Echo (ping) reply
19	4.351	192.168.1.10	8.8.8.8	ICMP	74	Echo (ping) request
20	4.362	8.8.8.8	192.168.1.10	ICMP	74	Echo (ping) repl

RESULT: Hence the working of wireshark and the simulation has been studied and verified.